# LAPORAN PRAKTIKUM
# PEMROGRAMAN PERANGKAT BERGERAK

## PERTEMUAN 8
## BOTTOM NAVIGATION VIEW, JETPACK NAVIGATION COMPONENT, DAN FRAGMENT





| | |
|---|---|
| **Nama** | : Given Gerald Chandra |
| **NIM** | : 25/560235/SV/26410 |
| **Kelas** | : PL3B2 [Rabu] |
| **Dosen Pengampu** | : Dr. Irkham Huda, S.Kom., M.Cs. |

**PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK**
**DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA**
**SEKOLAH VOKASI — UNIVERSITAS GADJAH MADA**
**YOGYAKARTA**

**2026**

# DAFTAR ISI

**Bab 1 Pendahuluan** **1**

1.1 Tujuan Praktikum 1

**Bab 2 Pembahasan Pertanyaan Assignment** **2**

2.1 Apa Itu Fragment? 2

2.2 Bagaimana Menampilkan Fragment ke dalam Activity? 3

2.3 Bagaimana Menambahkan Bottom Navigation? 4

2.4 Bagaimana Melakukan Navigasi dari Satu Fragment ke Fragment Lain? 5

2.5 Bagaimana Cara Berkomunikasi Antar Fragment? 6

**Bab 3 Hasil dan Pembahasan** **8**

3.1 Konfigurasi Proyek dan Dependensi Gradle 8

3.2 Penyiapan Aset Vektor dan File Menu Navigasi 9

3.3 Perancangan Navigation Graph XML 10

3.4 Tata Letak Utama dan Inisialisasi MainActivity 12

3.5 Implementasi Halaman Utama (HomeFragment) 13

3.6 Implementasi Halaman Checkout (CheckoutFragment) 15

3.7 Implementasi Halaman Pemilihan Alamat (AddressFragment) 17

3.8 Pengujian Halaman Utama dan Bilah Bottom Navigation 19

3.9 Pengujian Navigasi dan Pengiriman Argumen Safe Args 20

3.10 Pengujian Pemilihan Alamat pada Spinner 21

3.11 Pengujian Pengembalian Data Alamat via SavedStateHandle 22

3.12 Pengujian Alur Kembali ke Halaman Utama 23

**Bab 4 Kesimpulan** **24**

**Daftar Pustaka** **25**

# BAB 1
# PENDAHULUAN

## 1.1 Tujuan Praktikum

Tujuan praktikum Pemrograman Perangkat Bergerak Pertemuan 8 ini adalah:

1. Memahami konsep, karakteristik, dan siklus hidup (*lifecycle*) *Fragment* pada arsitektur aplikasi Android.
2. Menampilkan dan mengelola *Fragment* di dalam Activity menggunakan kontainer `NavHostFragment`.
3. Menghubungkan komponen `BottomNavigationView` dengan `NavController` agar navigasi antar-layar utama berjalan otomatis.
4. Merancang alur navigasi menggunakan file *Navigation Graph* XML (`bottom_navigation.xml`).
5. Menerapkan plugin *Safe Args* untuk navigasi antar-fragmen yang aman dari kesalahan tipe data (*type-safe*).
6. Mengirimkan dan mengembalikan data antar-fragmen menggunakan kombinasi argumen *Safe Args* dan `SavedStateHandle`.
7. Menyelesaikan pembuatan aplikasi e-commerce `MyShopGiven` sesuai seluruh instruksi modul praktikum.

Implementasi praktikum ini dikembangkan pada repositori GitHub publik: https://github.com/9iven/MyShopGiven.

# BAB 2
# PEMBAHASAN PERTANYAAN ASSIGNMENT

## 2.1 Apa Itu Fragment?

*Fragment* adalah bagian modular dari antarmuka pengguna (*user interface*) pada aplikasi Android yang memiliki siklus hidup sendiri dan menangani interaksi pengguna secara mandiri [1]. Fragment harus selalu berada di dalam Activity penampung (*host Activity*). Fragment tidak terdaftar pada `AndroidManifest.xml`.

Keuntungan memakai Fragment:

- **Modularitas:** Memecah kode antarmuka yang besar menjadi modul-modul kecil yang mandiri.
- **Reusabilitas:** Satu Fragment dapat digunakan ulang di berbagai layout atau Activity tanpa menulis ulang kode logika.
- **Mendukung Single-Activity:** Memudahkan implementasi aplikasi berbasis satu Activity utama yang menampung banyak layar fragmen.

Siklus hidup Fragment terdiri dari tahapan utama:

1. `onAttach()`: Fragment terhubung dengan Activity induk.
2. `onCreate()`: Inisialisasi data non-UI.
3. `onCreateView()`: Meng-*inflate* layout XML menjadi tampilan Fragment.
4. `onViewCreated()`: Menghubungkan View Binding, memasang event klik tombol, dan mengamati data.
5. `onStart()` dan `onResume()`: Tampilan Fragment aktif dan siap menerima interaksi pengguna.
6. `onPause()` dan `onStop()`: Fragment tertutup layar lain atau kehilangan fokus.
7. `onDestroyView()`: Tampilan Fragment dibersihkan. Di sini variabel binding disetel ke `null` untuk mencegah kebocoran memori (*memory leak*).
8. `onDestroy()` dan `onDetach()`: Fragment dilepas dari Activity penampung.

## 2.2 Bagaimana Menampilkan Fragment ke dalam Activity?

Ada dua cara untuk menampilkan Fragment ke dalam Activity:

### 1. Pendekatan Modern: NavHostFragment (Jetpack Navigation)

Cara ini direkomendasikan Google karena tidak memerlukan kode transaksi manual [2]. Pengembang cukup menaruh kontainer `NavHostFragment` pada file layout `activity_main.xml`:

**Listing 2.1: Deklarasi NavHostFragment pada layout**

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/nav_host_fragment"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    app:defaultNavHost="true"
    app:navGraph="@navigation/bottom_navigation"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintBottom_toTopOf="@id/bottom_navigation_view" />
```

Atribut `app:navGraph` mengarahkan kontainer ke file navigasi XML. Activity otomatis memuat layar pertama sesuai tag `startDestination`.

### 2. Pendekatan Manual: FragmentManager

Pada cara manual, Activity memakai `supportFragmentManager` untuk menjalankan transaksi:

**Listing 2.2: Transaksi manual via FragmentManager**

```
val homeFragment = HomeFragment()
supportFragmentManager.beginTransaction().apply {
    replace(R.id.fragment_container, homeFragment)
    addToBackStack(null)
    commit()
}
```

## 2.3 Bagaimana Menambahkan Bottom Navigation?

Empat langkah untuk memasang `BottomNavigationView` [3]:

1. **Pasang dependensi:** Tambahkan pustaka Material dan Navigation UI pada file `app/build.gradle.kts`:

   ```
   implementation(libs.material)
   val nav_version = "2.10.2"
   implementation("androidx.navigation:navigation-ui-ktx:$nav_version")
   ```

2. **Buat file menu XML:** Buat file `res/menu/bottom_navigation_menu.xml`. Nilai atribut `android:id` pada setiap item **harus sama persis** dengan ID fragment di `bottom_navigation.xml`:

   **Listing 2.3: File menu bottom_navigation_menu.xml**

   ```
   <menu xmlns:android="http://schemas.android.com/apk/res/android">
       <item
           android:id="@+id/homeFragment"
           android:icon="@drawable/baseline_home_24"
           android:title="Home" />
       <item
           android:id="@+id/cartFragment"
           android:icon="@drawable/baseline_shopping_cart_24"
           android:title="Keranjang" />
   ```

```
    <item
        android:id="@+id/profileFragment"
        android:icon="@drawable/baseline_person_24"
        android:title="Profile" />
</menu>
```

3. **Pasang elemen pada layout:** Letakkan `BottomNavigationView` di bagian bawah file layout `activity_main.xml`:

**Listing 2.4: Elemen BottomNavigationView pada activity_main.xml**

```
<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottom_navigation_view"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    app:layout_constraintBottom_toBottomOf="parent"
    app:menu="@menu/bottom_navigation_menu" />
```

4. **Hubungkan dengan NavController:** Di dalam file `MainActivity.kt`, panggil fungsi `setupWithNavController`:

**Listing 2.5: Menghubungkan BottomNavigationView dengan NavController**

```
val navController = findNavController(R.id.nav_host_fragment)
binding.bottomNavigationView.setupWithNavController(navController)
```

Fungsi ini otomatis mengubah tampilan Fragment saat menu ditekan dan mengelola status tumpukan layar.

## 2.4 Bagaimana Melakukan Navigasi dari Satu Fragment ke Fragment Lain?

Navigasi antar-fragmen dilakukan dengan menghubungkan dua layar memakai tag `<action>` pada file grafik navigasi, lalu mengeksekusinya di kode Kotlin memakai kelas *Directions* dari plugin *Safe Args* [4].

### 1. Definisi Action pada File Navigasi XML

Pada file `bottom_navigation.xml`, buat tag `<action>` di dalam fragmen asal:

**Listing 2.6: Definisi aksi navigasi pada bottom_navigation.xml**

```
<fragment
    android:id="@+id/homeFragment"
    android:name="com.vokasi.myshopgiven.HomeFragment"
    android:label="fragment_home"
    tools:layout="@layout/fragment_home" >
    <action
        android:id="@+id/action_homeFragment_to_checkoutFragment"
        app:destination="@id/checkoutFragment" />
</fragment>
```

### 2. Eksekusi Navigasi pada Kode Kotlin

Pada file `HomeFragment.kt`, navigasi dipanggil saat tombol ditekan:

**Listing 2.7: Eksekusi navigasi pada HomeFragment.kt**

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    with(binding) {
        btnBuy.setOnClickListener {
            val action = HomeFragmentDirections
                .actionHomeFragmentToCheckoutFragment(txtProduct.text.toString())
            findNavController().navigate(action)
        }
    }
}
```

Penjelasan baris kode:

- `HomeFragmentDirections` adalah kelas pembantu otomatis hasil kompilasi Safe Args. Fungsi `actionHomeFragmentToCheckoutFragment` membuat objek navigasi dan memasukkan teks nama produk sebagai argumen.
- `findNavController().navigate(action)` mengeksekusi perpindahan layar menuju `CheckoutFragment` dan menyimpan posisi layar asal ke dalam tumpukan navigasi (*back stack*).

## 2.5 Bagaimana Cara Berkomunikasi Antar Fragment?

Ada dua alur komunikasi antar-fragmen:

### 1. Mengirim Data ke Layar Tujuan (Safe Args)

Digunakan saat Fragment A berpindah ke Fragment B sambil membawa data:

1. Tentukan parameter data pada file `bottom_navigation.xml`:

```
<fragment
    android:id="@+id/checkoutFragment"
    android:name="com.vokasi.myshopgiven.CheckoutFragment" >
    <argument
        android:name="productName"
        app:argType="string"
        app:nullable="true" />
</fragment>
```

2. Kirim data dari Fragment asal lewat aksi navigasi:

```
val action = HomeFragmentDirections.actionHomeFragmentToCheckoutFragment("Samsung M2
2")
findNavController().navigate(action)
```

3. Tangkap data di Fragment tujuan memakai delegasi `navArgs()`:

**Listing 2.8: Mengekstrak data memakai navArgs()**

```
class CheckoutFragment : Fragment() {
    private val args: CheckoutFragmentArgs by navArgs()

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        binding.txtProductName.text = args.productName
    }
}
```

**2. Mengembalikan Data ke Layar Sebelumnya (SavedStateHandle)**

Digunakan saat Fragment B (misal `AddressFragment`) dibuka untuk memilih data, lalu hasilnya dikirim kembali ke Fragment A (`CheckoutFragment`) saat tombol selesai ditekan [5]:

1. Fragment pengirim menyimpan data pada `previousBackStackEntry` lalu memanggil `navigateUp()`:

   **Listing 2.9: Menyimpan data hasil pada AddressFragment.kt**

   ```
   binding.btnDone.setOnClickListener {
       val selectedProvince = binding.spinnerProvinces.selectedItem.toString()
       findNavController().apply {
           previousBackStackEntry
               ?.savedStateHandle
               ?.set("address", selectedProvince)
       }.navigateUp()
   }
   ```

2. Fragment penerima mengamati data kembalian lewat `LiveData`:

   **Listing 2.10: Mengamati data hasil pada CheckoutFragment.kt**

   ```
   findNavController().currentBackStackEntry?.savedStateHandle?.let { handle ->
       handle.getLiveData<String>("address")
           .observe(viewLifecycleOwner) { res ->
               binding.edtAddress.setText(res)
           }
   }
   ```

# BAB 3
# HASIL DAN PEMBAHASAN

## 3.1 Konfigurasi Proyek dan Dependensi Gradle

Pengembangan aplikasi `MyShopGiven` memakai target SDK 36 dan minimum SDK 24. Plugin *Safe Args* dikonfigurasi pada file `settings.gradle.kts` (Listing 3.1).

**Listing 3.1: Konfigurasi Safe Args pada settings.gradle.kts**

```
pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
    plugins {
        id("androidx.navigation.safeargs.kotlin") version "2.10.2"
    }
}
rootProject.name = "MyShopGiven"
include(":app")
```

Pada file `app/build.gradle.kts`, fitur View Binding diaktifkan dan dependensi Navigation Component didaftarkan (Listing 3.2).

**Listing 3.2: Konfigurasi View Binding dan dependensi pada app/build.gradle.kts**

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    id("androidx.navigation.safeargs.kotlin")
}

android {
    namespace = "com.vokasi.myshopgiven"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.vokasi.myshopgiven"
        minSdk = 24
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildFeatures {
        viewBinding = true
    }
}

dependencies {
```

```
    implementation("androidx.core:core-ktx:1.15.0")
    implementation(libs.androidx.appcompat)
    implementation(libs.material)
    implementation(libs.androidx.activity)
    implementation(libs.androidx.constraintlayout)

    val nav_version = "2.10.2"
    implementation("androidx.navigation:navigation-fragment-ktx:$nav_version")
    implementation("androidx.navigation:navigation-ui-ktx:$nav_version")
}
```

- Pada listing 3.1 baris 7–9, blok `pluginManagement` mendaftarkan repositori `google()`, `mavenCentral()`, dan `gradlePluginPortal()` untuk mengunduh plugin Gradle.
- Pada baris 11, plugin `androidx.navigation.safeargs.kotlin` versi 2.10.2 didaftarkan agar Gradle dapat men-generate class Directions dan Args untuk navigasi type-safe.
- Pada listing 3.2 baris 4, plugin `androidx.navigation.safeargs.kotlin` diterapkan pada modul aplikasi.
- Pada baris 9–10, `compileSdk` dan `targetSdk` disetel pada level 36, dengan `minSdk` 24 untuk mendukung kompatibilitas perangkat modern.
- Pada baris 21–23, blok `buildFeatures` mengaktifkan `viewBinding = true` untuk mempermudah manipulasi elemen UI tanpa `findViewById()`.
- Pada baris 33–34, pustaka `navigation-fragment-ktx` dan `navigation-ui-ktx` versi 2.10.2 ditambahkan sebagai dependensi navigasi Jetpack.

## 3.2 Penyiapan Aset Vektor dan File Menu Navigasi

### Pemakaian Aset Vektor Bawaan (Rename Vector Asset)

Ikon navigasi dan gambar produk tidak diambil dari menu pencarian Clip Art eksternal, melainkan memakai aset vektor bawaan Android Studio yang diganti namanya (*rename*) menjadi:

- `res/drawable/baseline_home_24.xml` (ikon beranda)
- `res/drawable/baseline_shopping_cart_24.xml` (ikon keranjang)
- `res/drawable/baseline_person_24.xml` (ikon profil)
- `res/drawable/baseline_smartphone_24.xml` (ikon produk Samsung M22)

File menu navigasi bawah dibuat pada file `res/menu/bottom_navigation_menu.xml` (Listing 3.3).

**Listing 3.3: File res/menu/bottom_navigation_menu.xml**

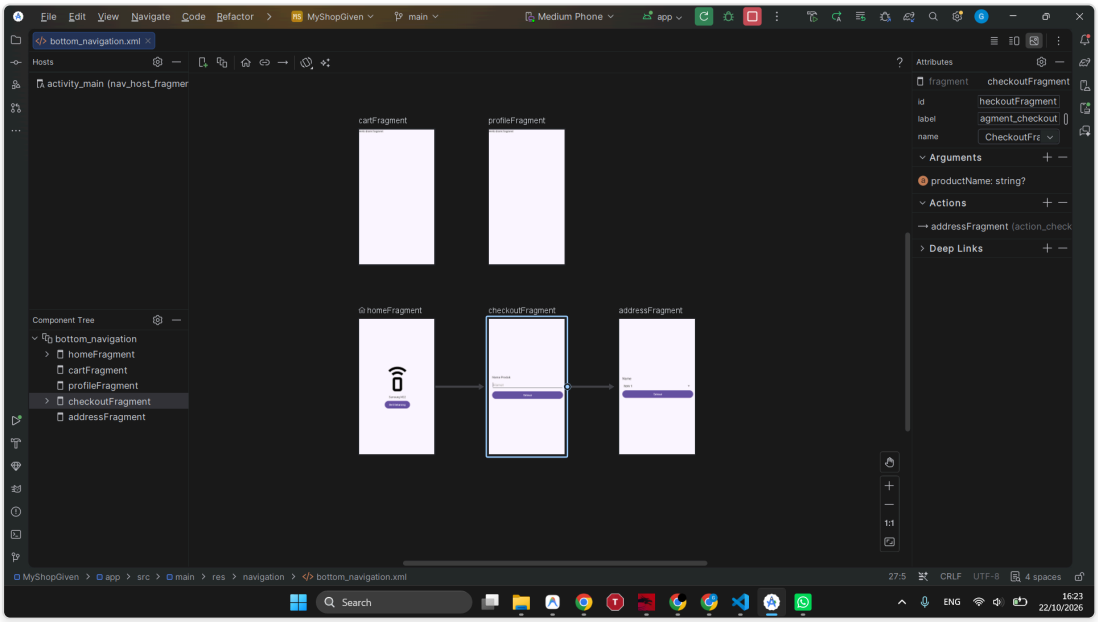
```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/homeFragment"
        android:icon="@drawable/baseline_home_24"
        android:title="Home" />
    <item
        android:id="@+id/cartFragment"
```

```
        android:icon="@drawable/baseline_shopping_cart_24"
        android:title="Keranjang" />
    <item
        android:id="@+id/profileFragment"
        android:icon="@drawable/baseline_person_24"
        android:title="Profile" />
</menu>
```

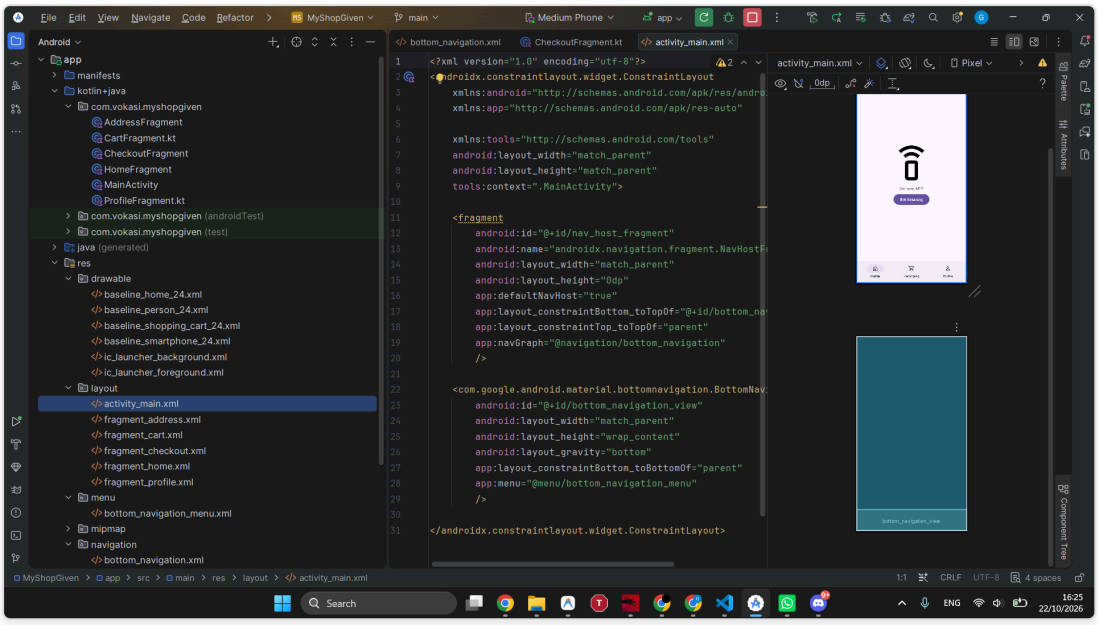
- Pada listing 3.3 baris 2–5, item `homeFragment` didefinisikan dengan ikon `baseline_home_24` dan label "Home". ID ini sama persis dengan ID tujuan pada navigation graph.
- Pada baris 6–9, item `cartFragment` menggunakan ikon `baseline_shopping_cart_24` dengan label "Keranjang".
- Pada baris 10–13, item `profileFragment` menggunakan ikon `baseline_person_24` dengan label "Profile".

## 3.3 Perancangan Navigation Graph XML

Pada direktori `res/navigation/`, dibuat file `bottom_navigation.xml` untuk mendefinisikan lima fragmen: `homeFragment`, `cartFragment`, `profileFragment`, `checkoutFragment`, dan `addressFragment`. Diagram alur navigasi dari Android Studio laptop praktikan ditampilkan pada Gambar 3.1 dan Gambar 3.2.




*Gambar 3.1: Tampilan diagram Navigation Graph pada editor Android Studio laptop praktikan*



*Gambar 3.2: Tampilan jendela kerja Android Studio laptop praktikan memperlihatkan struktur proyek*

Berikut isi lengkap file `bottom_navigation.xml` pada Listing 3.4.

**Listing 3.4: File res/navigation/bottom_navigation.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/bottom_navigation"
    app:startDestination="@id/homeFragment">

    <fragment
        android:id="@+id/homeFragment"
        android:name="com.vokasi.myshopgiven.HomeFragment"
        android:label="fragment_home"
        tools:layout="@layout/fragment_home" >
        <action
            android:id="@+id/action_homeFragment_to_checkoutFragment"
            app:destination="@id/checkoutFragment" />
    </fragment>

    <fragment
        android:id="@+id/cartFragment"
        android:name="com.vokasi.myshopgiven.CartFragment"
        android:label="fragment_cart"
        tools:layout="@layout/fragment_cart" />

    <fragment
        android:id="@+id/profileFragment"
        android:name="com.vokasi.myshopgiven.ProfileFragment"
        android:label="fragment_profile"
        tools:layout="@layout/fragment_profile" />

    <fragment
        android:id="@+id/checkoutFragment"
        android:name="com.vokasi.myshopgiven.CheckoutFragment"
```

```
        android:label="fragment_checkout"
        tools:layout="@layout/fragment_checkout" >
        <argument
            android:name="productName"
            app:argType="string"
            app:nullable="true" />
        <action
            android:id="@+id/action_checkoutFragment_to_addressFragment"
            app:destination="@id/addressFragment" />
    </fragment>

    <fragment
        android:id="@+id/addressFragment"
        android:name="com.vokasi.myshopgiven.AddressFragment"
        android:label="fragment_address"
        tools:layout="@layout/fragment_address" />
</navigation>
```

- Pada listing 3.4 baris 6, tag `navigation` menetapkan `startDestination` pada `@id/homeFragment` sebagai layar pertama yang tampil.
- Pada baris 8–17, fragmen `homeFragment` didaftarkan dengan aksi `action_homeFragment_to_checkoutFragment` menuju `checkoutFragment`.
- Pada baris 19–29, fragmen `cartFragment` dan `profileFragment` didaftarkan sebagai destinasi tingkat atas yang terhubung langsung dengan bilah navigasi bawah.
- Pada baris 31–43, fragmen `checkoutFragment` menerima argumen `productName` bertipe string (nullable = true), serta aksi `action_checkoutFragment_to_addressFragment` menuju `addressFragment`.
- Pada baris 45–49, fragmen `addressFragment` didefinisikan sebagai destinasi pemilihan alamat.
- Tampilan visual grafik navigasi pada editor Android Studio (Gambar 3.1) dan struktur proyek (Gambar 3.2) memverifikasi keterhubungan seluruh fragmen secara grafis.

## 3.4 Tata Letak Utama dan Inisialisasi MainActivity

File `activity_main.xml` menempatkan `NavHostFragment` di area atas dan `BottomNavigationView` menempel di bawah (Listing 3.5).

**Listing 3.5: Tata letak activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <fragment
        android:id="@+id/nav_host_fragment"
```

```
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        app:defaultNavHost="true"
        app:layout_constraintBottom_toTopOf="@+id/bottom_navigation_view"
        app:layout_constraintTop_toTopOf="parent"
        app:navGraph="@navigation/bottom_navigation" />

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_navigation_view"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom"
        app:layout_constraintBottom_toBottomOf="parent"
        app:menu="@menu/bottom_navigation_menu" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

File `MainActivity.kt` menghubungkan NavController ke bilah navigasi bawah (Listing 3.6).

**Listing 3.6: File MainActivity.kt**

```
package com.vokasi.myshopgiven

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.navigation.findNavController
import androidx.navigation.ui.setupWithNavController
import com.vokasi.myshopgiven.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        with(binding) {
            val navController = findNavController(R.id.nav_host_fragment)
            bottomNavigationView.setupWithNavController(navController)
        }
    }
}
```

- Pada listing 3.5 baris 8–16, kontainer `NavHostFragment` menempati area atas layar dengan atribut `defaultNavHost="true"` dan `navGraph="@navigation/bottom_navigation"`.
- Pada baris 18–25, elemen `BottomNavigationView` diletakkan menempel pada bagian bawah layar dengan atribut `menu="@menu/bottom_navigation_menu"`.
- Pada listing 3.6 baris 12, variabel `binding` diinisialisasi melalui `ActivityMainBinding.inflate(layoutInflater)` dan disetel sebagai contentView.
- Pada baris 16, fungsi `findNavController(R.id.nav_host_fragment)` mengambil instance NavController dari kontainer NavHostFragment.

- Pada baris 17, metode `setupWithNavController()` menghubungkan BottomNavigationView dengan NavController agar pergantian tab menu mengalihkan fragmen secara otomatis.

## 3.5 Implementasi Halaman Utama (HomeFragment)

File `fragment_home.xml` memuat gambar smartphone, label Samsung M22, dan tombol Beli Sekarang (Listing 3.7).

**Listing 3.7: Tata letak fragment_home.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".HomeFragment">

    <ImageView
        android:layout_width="150dp"
        android:layout_height="150dp"
        android:src="@drawable/baseline_smartphone_24"/>

    <TextView
        android:id="@+id/txtProduct"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="14sp"
        android:layout_marginVertical="8dp"
        android:text="Samsung M22"/>

    <Button
        android:id="@+id/btn_buy"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Beli Sekarang"/>
</LinearLayout>
```

Saat tombol Beli Sekarang ditekan, `HomeFragment.kt` mengirimkan nama produk menuju `CheckoutFragment` (Listing 3.8).

**Listing 3.8: File HomeFragment.kt**

```
package com.vokasi.myshopgiven

import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.navigation.fragment.findNavController
import com.vokasi.myshopgiven.databinding.FragmentHomeBinding
```

```
class HomeFragment : Fragment() {
    private var _binding: FragmentHomeBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentHomeBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        with(binding) {
            val action = HomeFragmentDirections
                .actionHomeFragmentToCheckoutFragment(txtProduct.text.toString())
            btnBuy.setOnClickListener {
                findNavController().navigate(action)
            }
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

- Pada listing 3.7 baris 9–12, ImageView menampilkan ikon produk smartphone menggunakan aset drawable `baseline_smartphone_24`.
- Pada baris 14–20, TextView dengan ID `txtProduct` menampilkan nama produk "Samsung M22".
- Pada baris 22–26, Button dengan ID `btn_buy` disediakan untuk memicu perpindahan layar ke halaman checkout.
- Pada listing 3.8 baris 13–14, variabel `_binding` bertipe `FragmentHomeBinding?` diinisialisasi untuk mendukung lifecycle View Binding pada fragmen.
- Pada baris 20–23, metode `onCreateView()` meng-inflate layout fragment_home.xml dan mengembalikan root tampilan.
- Pada baris 29–34, metode `onViewCreated()` memasang click listener pada btnBuy. Class `HomeFragmentDirections` membuat objek action dengan argumen teks nama produk, lalu dieksekusi menggunakan `findNavController().navigate(action)`.
- Pada baris 38–41, metode `onDestroyView()` menyetel `_binding = null` untuk mencegah kebocoran memori saat tampilan fragmen dihancurkan.

### 3.6 Implementasi Halaman Checkout (CheckoutFragment)

File `fragment_checkout.xml` menampung kolom nama produk, kolom isian alamat (`clickable="false"` dan `focusable="false"`), dan tombol Selesai (Listing 3.9).

**Listing 3.9: Tata letak fragment_checkout.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".CheckoutFragment"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/txt_product_name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp"
        android:textSize="16sp"
        tools:text="Nama Produk" />

    <EditText
        android:id="@+id/edt_address"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp"
        android:hint="Alamat"
        android:inputType="textPostalAddress"
        android:clickable="false"
        android:focusable="false"
        android:longClickable="false"
        android:textIsSelectable="false" />

    <Button
        android:id="@+id/btn_done"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Selesai" />
</LinearLayout>
```

File `CheckoutFragment.kt` menampilkan argumen `productName`, membuka `AddressFragment` saat kolom alamat diklik, menyimak kembalian alamat lewat `SavedStateHandle`, dan kembali ke Home saat tombol Selesai diklik (Listing 3.10).

**Listing 3.10: File CheckoutFragment.kt**

```
package com.vokasi.myshopgiven

import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.navigation.fragment.findNavController
import androidx.navigation.fragment.navArgs
import com.vokasi.myshopgiven.databinding.FragmentCheckoutBinding

class CheckoutFragment : Fragment() {
```

```
    private var _binding: FragmentCheckoutBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentCheckoutBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        with(binding) {
            val args: CheckoutFragmentArgs by navArgs()
            txtProductName.text = args.productName

            edtAddress.setOnClickListener {
                val action = CheckoutFragmentDirections
                    .actionCheckoutFragmentToAddressFragment()
                findNavController().navigate(action)
            }

            findNavController().currentBackStackEntry?.savedStateHandle?.let { handle ->
                handle.getLiveData<String>("address")
                    .observe(viewLifecycleOwner) { res ->
                        edtAddress.setText(res)
                    }
            }

            btnDone.setOnClickListener {
                findNavController().navigateUp()
            }
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

- Pada listing 3.9 baris 9–15, TextView `txt_product_name` disiapkan untuk menampilkan teks nama produk yang dikirim dari HomeFragment.
- Pada baris 17–27, EditText `edt_address` dikonfigurasi dengan `clickable="false"` dan `focusable="false"` agar bertindak sebagai tombol pembuka halaman pemilihan alamat tanpa memicu keyboard virtual.
- Pada baris 29–33, Button `btn_done` disiapkan untuk menyelesaikan proses checkout dan kembali ke layar awal.
- Pada listing 3.10 baris 25, delegasi `navArgs()` mengekstrak argumen yang dikirim via Safe Args ke dalam variabel args.

- Pada baris 26, `txtProductName.text` diisi dengan nilai `args.productName` yang diterima dari HomeFragment.
- Pada baris 28–32, click listener pada edtAddress membuat aksi `CheckoutFragmentDirections.actionCheckoutFragmentToAddressFragment()` dan berpindah ke AddressFragment.
- Pada baris 34–39, `currentBackStackEntry?.savedStateHandle` mengamati key "address" secara reaktif melalui LiveData. Saat data baru tersedia, teks alamat otomatis dimasukkan ke edtAddress.
- Pada baris 41–43, click listener pada btnDone memanggil `findNavController().navigateUp()` untuk kembali ke layar HomeFragment.

## 3.7 Implementasi Halaman Pemilihan Alamat (AddressFragment)

Daftar provinsi didefinisikan pada file `res/values/strings.xml` (Listing 3.11).

**Listing 3.11: Array provinsi pada strings.xml**

```
<resources>
    <string name="app_name">MyShopGiven</string>

    <array name="provinces">
        <item>Aceh</item>
        <item>Bali</item>
        <item>Banten</item>
        <item>Bengkulu</item>
        <item>Gorontalo</item>
        <item>Jakarta</item>
        <item>Jambi</item>
        <item>Jawa Barat</item>
        <item>Jawa Tengah</item>
        <item>Jawa Timur</item>
    </array>
</resources>
```

File layout `fragment_address.xml` memuat komponen `Spinner` dan tombol Selesai (Listing 3.12).

**Listing 3.12: Tata letak fragment_address.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="16dp"
    tools:context=".AddressFragment">

    <TextView
        android:id="@+id/txt_name"
        android:layout_width="match_parent"
```

```
        android:textSize="18sp"
        tools:text="Name"
        android:layout_height="wrap_content"/>

    <Spinner
        android:id="@+id/spinnerProvinces"
        android:layout_marginTop="16dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp"/>

    <Button
        android:id="@+id/btn_done"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Selesai" />
</LinearLayout>
```

File `AddressFragment.kt` memasang data ke Spinner. Saat tombol Selesai diklik, nilai provinsi dikirim ke `previousBackStackEntry.savedStateHandle` lalu Fragment ditutup (Listing 3.13).

**Listing 3.13: File AddressFragment.kt**

```
package com.vokasi.myshopgiven

import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.ArrayAdapter
import androidx.navigation.fragment.findNavController
import com.vokasi.myshopgiven.databinding.FragmentAddressBinding

class AddressFragment : Fragment() {
    private var _binding: FragmentAddressBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentAddressBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        with(binding) {
            val provinces = resources.getStringArray(R.array.provinces)

            val adapterProvinces = ArrayAdapter(
                requireContext(),
                android.R.layout.simple_spinner_item,
                provinces
            )
            adapterProvinces.setDropDownViewResource(android.R.layout.simple_spinner_dropdo
```

```
wn_item)
            spinnerProvinces.adapter = adapterProvinces

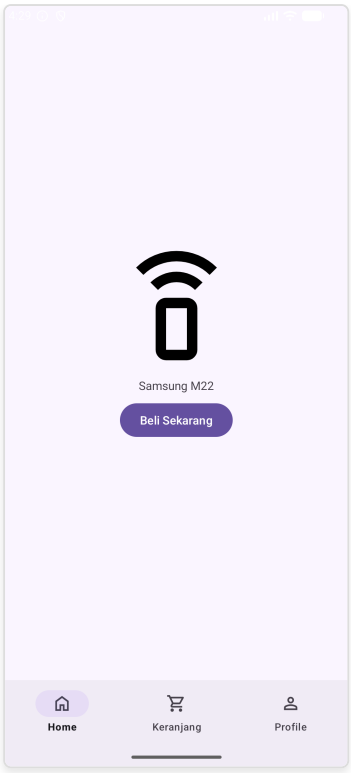
            btnDone.setOnClickListener {
                findNavController().apply {
                    previousBackStackEntry
                        ?.savedStateHandle?.set("address", spinnerProvinces.selectedItem.to
String())
                }.navigateUp()
            }
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

- Pada listing 3.11 baris 4–15, array string bernama provinces didefinisikan berisi 10 nama provinsi di Indonesia untuk sumber data Spinner.
- Pada listing 3.12 baris 9–14, TextView `txt_name` menampilkan label judul pemilihan alamat.
- Pada baris 16–21, komponen Spinner `spinnerProvinces` disiapkan untuk menampilkan daftar opsi provinsi.
- Pada baris 23–27, Button `btn_done` disediakan untuk mengonfirmasi pilihan provinsi dan kembali ke CheckoutFragment.
- Pada listing 3.13 baris 24, daftar provinsi diambil dari sumber daya strings.xml menggunakan `resources.getStringArray(R.array.provinces)`.
- Pada baris 26–32, ArrayAdapter dibuat dengan layout simple_spinner_item dan simple_spinner_dropdown_item, kemudian dipasang ke spinnerProvinces.
- Pada baris 34–40, click listener pada btnDone mengambil teks provinsi terpilih, menyimpannya ke `previousBackStackEntry?.savedStateHandle` dengan key "address", lalu memanggil `navigateUp()` untuk menutup AddressFragment.

## 3.8 Pengujian Halaman Utama dan Bilah Bottom Navigation

Pengujian aplikasi `MyShopGiven` dijalankan langsung pada emulator *Medium Phone* di Android Studio laptop praktikan. Tampilan awal aplikasi saat pertama kali diluncurkan disajikan pada Gambar 3.3.

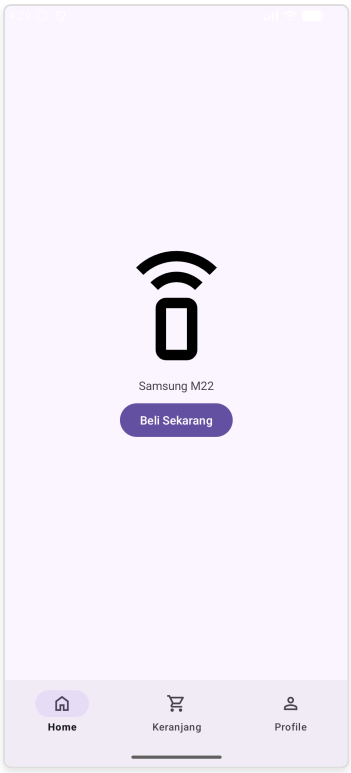



*Gambar 3.3: Tampilan Halaman Home Aktif dengan Bottom Navigation pada Emulator*

- Pada Gambar 3.3, aplikasi MyShopGiven berhasil dijalankan pada emulator Medium Phone dan menampilkan fragmen Home sebagai `startDestination`.
- Bilah BottomNavigationView di bagian bawah menampilkan tiga tab: Home, Keranjang, dan Profile sesuai konfigurasi menu.
- Halaman memuat gambar smartphone, label produk "Samsung M22", dan tombol "Beli Sekarang".

## 3.9 Pengujian Navigasi dan Pengiriman Argumen Safe Args

Pengujian perpindahan layar dilakukan dengan menekan tombol "Beli Sekarang" pada halaman Home. Hasil tampilan halaman Checkout disajikan pada Gambar 3.4.

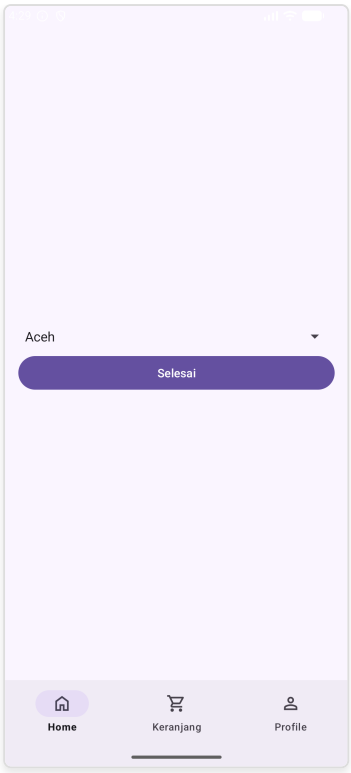



*Gambar 3.4: Tampilan Halaman Checkout Menerima Data Argumen Produk via Safe Args*

- Saat tombol Beli Sekarang pada layar Home ditekan, NavController mengeksekusi aksi `action_homeFragment_to_checkoutFragment` (Gambar 3.4).
- Data teks nama produk ("Samsung M22") terkirim secara type-safe melalui plugin Safe Args dan otomatis ditampilkan pada TextView `txt_product_name`.
- Kolom isian alamat dalam keadaan kosong dan siap untuk diklik guna memilih lokasi pengiriman.

## 3.10 Pengujian Pemilihan Alamat pada Spinner

Pengujian pemilihan alamat dilakukan dengan mengklik kolom isian alamat pada halaman Checkout. Hasil tampilan halaman AddressFragment disajikan pada Gambar 3.5.

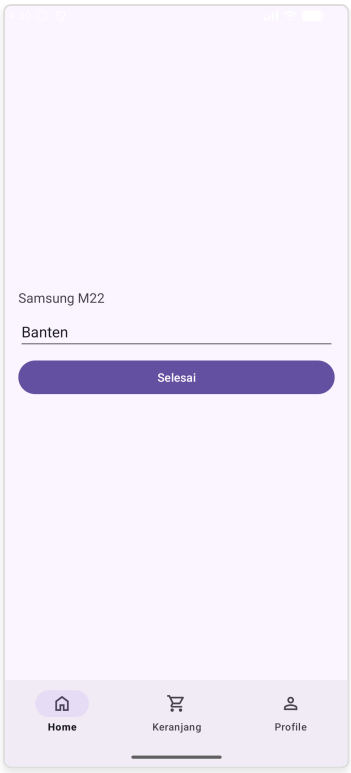



*Gambar 3.5: Tampilan Halaman Pemilihan Alamat dengan Dropdown Spinner Provinsi*

- Saat kolom isian alamat pada halaman Checkout diklik, aplikasi membuka AddressFragment (Gambar 3.5).
- Komponen Spinner berhasil memuat array provinsi dari strings.xml melalui ArrayAdapter.
- Pengguna dapat memilih salah satu provinsi dari daftar opsi yang tersedia, misalnya "Jawa Barat".

## 3.11 Pengujian Pengembalian Data Alamat via SavedStateHandle

Pengujian pengembalian data dilakukan setelah pengguna memilih provinsi dan menekan tombol "Selesai" pada AddressFragment. Hasil tampilan halaman Checkout disajikan pada Gambar 3.6.

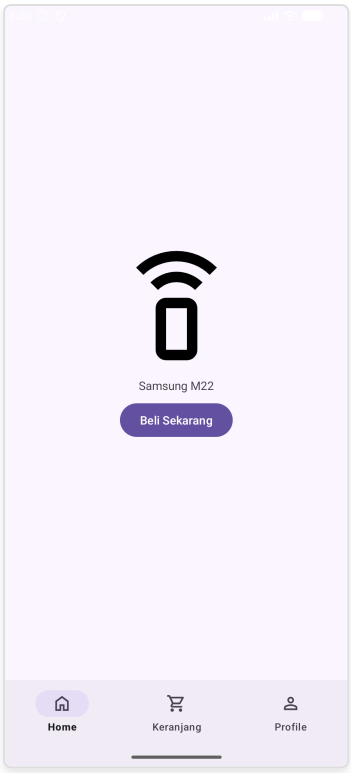



*Gambar 3.6: Tampilan Halaman Checkout dengan Alamat yang Telah Terisi via SavedStateHandle*

- Setelah provinsi dipilih dan tombol Selesai pada AddressFragment ditekan, data provinsi disimpan ke `savedStateHandle` pada `previousBackStackEntry`.
- Aplikasi kembali ke CheckoutFragment dan observer LiveData langsung mendeteksi data kembalian, lalu mengisikannya ke kolom isian alamat (Gambar 3.6).
- Pengujian ini membuktikan bahwa mekanisme two-way communication antar-fragmen menggunakan SavedStateHandle berjalan tanpa kendala.

## 3.12 Pengujian Alur Kembali ke Halaman Utama

Pengujian alur akhir dilakukan dengan menekan tombol "Selesai" pada CheckoutFragment untuk menyelesaikan transaksi belanja. Hasil tampilan disajikan pada Gambar 3.7.




*Gambar 3.7: Tampilan Aplikasi Kembali ke Layar Home Setelah Proses Selesai*

- Saat tombol Selesai pada CheckoutFragment ditekan, fungsi `findNavController().navigateUp()` dipanggil (Gambar 3.7).
- Aplikasi kembali ke halaman awal (HomeFragment) dan kondisi back stack dibersihkan secara rapi.
- Pengujian membuktikan seluruh siklus transaksi dan navigasi aplikasi MyShopGiven telah berhasil diselesaikan sesuai spesifikasi modul praktikum.

# BAB 4
# KESIMPULAN

Berdasarkan perancangan dan pengujian aplikasi `MyShopGiven` pada praktikum Pemrograman Perangkat Bergerak Pertemuan 8, kesimpulan yang diperoleh adalah:

1. Fragment memecah antarmuka menjadi komponen modular yang mandiri dan dapat digunakan ulang pada arsitektur Single-Activity.
2. Pemakaian aset vektor bawaan Android Studio yang di-*rename* memudahkan penyiapan ikon visual yang tajam tanpa bergantung pada aset eksternal.
3. Fungsi `setupWithNavController` menghubungkan `BottomNavigationView` dengan `NavController` secara otomatis tanpa perlu transaksi fragmen manual.
4. File *Navigation Graph* XML memetakan alur antar-layar dan aksi navigasi secara terstruktur.
5. Plugin *Safe Args* menjamin pengiriman parameter antar-fragmen aman dari kesalahan tipe data saat kompilasi.
6. Mekanisme `SavedStateHandle` mengembalikan data hasil pilihan alamat dari `AddressFragment` ke `CheckoutFragment` secara reaktif lewat `LiveData`.
7. Seluruh alur aplikasi `MyShopGiven` selesai sesuai modul praktikum dan tersimpan pada repositori GitHub resmi.

# DAFTAR PUSTAKA


1. Google Developers, "Fragments | Android Developers," *Android Open Source Project*, 2025. [Online]. Tersedia: https://developer.android.com/guide/fragments. [Diakses: 22 Oktober 2026].
2. Google Developers, "Get started with the Navigation component | Android Developers," *Android Open Source Project*, 2025. [Online]. Tersedia: https://developer.android.com/guide/navigation/get-started. [Diakses: 22 Oktober 2026].
3. Google Developers, "Bottom navigation - Material Design," *Material Design*, 2025. [Online]. Tersedia: https://m3.material.io/components/bottom-navigation/overview. [Diakses: 22 Oktober 2026].
4. Google Developers, "Pass data between destinations with Safe Args | Android Developers," *Android Open Source Project*, 2025. [Online]. Tersedia: https://developer.android.com/guide/navigation/use-graph/pass-data#Safe-args. [Diakses: 22 Oktober 2026].
5. Google Developers, "Return a result from a destination | Android Developers," *Android Open Source Project*, 2025. [Online]. Tersedia: https://developer.android.com/guide/navigation/use-graph/programmatic#return_a_result. [Diakses: 22 Oktober 2026].